

Project Proposal

TrackServ: A Web-Based Service Reporting and Tracking Platform

Group Name: **TrackServ**

Group No.: 04

Date: 10 August 2026 (Post-implementation review, citizen portal and admin portal)

Abstract

TrackServ is an open, transparent, web-based platform designed to bridge the communication gap between local municipalities and their communities. What was originally scoped as a single reporting-and-tracking portal was implemented as a two-sided civic engagement system: a public-facing citizen portal for reporting and following issues, and a separate, purpose-built administrative dashboard for municipal staff to triage, assign, and analyse those reports. The citizen side delivers guided issue reporting with geolocation, reverse geocoding and municipal-boundary validation, a real-time personal dashboard, community upvoting, and a badge system. The administrative side delivers a live operations map, filterable report management, a staff assignment and workload system, and an analytics module tracking resolution time, resolution rate and overdue reports. Both applications share a Supabase (PostgreSQL) backend and stay in sync through Supabase's real-time change-data-capture subscriptions, so status changes made by an administrator appear on the citizen's dashboard within seconds.

Contents

Abstract	1
1. Introduction	3
2. Problem Definition and Aims	3
2.1 Client needs and design criteria / Problem statement	3
2.2 Existing work	3
2.3 Proposed solution	3
3. Design considerations and specifications	4
3.1 Technologies involved	4
3.2 Hardware and software limitations	4
3.3 Performance limitations	4
4. Required skills	5
5. Design approach/methodology	5
5.1 Methodology	5
5.2 Project components	5
5.3 Planned timeline	5
6. Architecture & Technology Decisions	6
7. Citizen Portal Features	6
7.1 Reporting an issue	6
7.2 Tracking progress	6
7.3 Community voting and badges	6
7.4 Responsive design	6
8. Administrative Dashboard	7
8.1 Operations map and situational awareness	7
8.2 Filtering and prioritisation	7
8.3 Report management	7
8.4 Assignment system	7
8.5 Analytics	7
8.6 Manual report entry	7
9. Real-Time Synchronisation Between Portals	8
10. Media & File Management	8
11. Security Assessment	8
12. Performance & Scalability	8
13. Testing & QA	9
14. Evaluation of solution	9
15. Known Limitations & Future Work	9
16. Production Readiness Roadmap	9
17. Conclusion	10
References	11

1. Introduction

There is currently a significant communication gap between local municipalities and the communities they serve. Issues requiring municipal attention, such as infrastructure damage, utility outages, or sanitation concerns, often go unreported because citizens lack an accessible, centralised reporting channel. Furthermore, even when reports are made through traditional means, citizens are frequently left unaware of the progress or resolution status, leading to a breakdown in trust and public frustration.

TrackServ was proposed as an open, transparent, web-based platform designed to bridge this gap. Implementation showed that solving this problem well required attention to both ends of the reporting relationship: citizens need a simple, trustworthy way to report and follow up, while the municipality needs an operational tool for triaging, assigning and measuring its own response. The delivered system therefore split into two connected applications, a citizen portal and an administrative dashboard kept in sync in real time, alongside the community voting and recognition features introduced to sustain citizen participation beyond a single report.

2. Problem Definition and Aims

2.1 Client needs and design criteria / Problem statement

The primary stakeholders (the municipality and the community) require a system that addresses the following:

- **Communication Inefficiency:** A lack of a direct, digital pipeline for citizens to report issues.
- **Transparency Gaps:** Citizens have no way to track if a reported issue is being addressed.
- **Data Accuracy:** Difficulty for the municipality to pinpoint the exact location of reported civic issues without visual or GPS data.
- **Sustained Engagement** (identified during implementation): without visible community support for a report or recognition for participating, one-off reporters were unlikely to return.
- **Operational Triage** (identified during implementation): the municipality side needed its own tool to filter, assign, and measure its response to reports a need the original citizen-facing scope did not cover.

2.2 Existing work

Current solutions often rely on manual reporting, phone calls, or emails, which are difficult to track and lack transparency. While some municipalities have basic web forms, they often lack a real-time status dashboard or integrated geolocation mapping for the user for example the e-tshwane.co.za website (madiba, et al., 2023).

2.3 Proposed solution

TrackServ delivers a centralised citizen portal where issues are logged with an automatically captured and adjustable GPS location, validated against municipal boundaries, and tracked on a personal dashboard. It is paired with a dedicated administrative dashboard where municipal staff view all reports on a live map, filter and search them, assign them to team members, and monitor resolution performance through analytics. A community voting system lets citizens support existing reports instead of duplicating them, and a badge system rewards continued participation. Together, the two applications form a closed loop: citizen reports flow into the admin dashboard in real time, and administrative status changes flow back to the citizen dashboard in real time.

3. Design considerations and specifications

3.1 Technologies involved

The proposal originally specified a custom multi-tier stack: React.js/Node.js on the frontend and application layer, PHP for API development, and a MySQL relational database. During implementation this was replaced with Supabase, a managed backend-as-a-service built on PostgreSQL:

- PostgreSQL (via Supabase) replaces the proposed MySQL database.
- Supabase Auth replaces bespoke authentication logic for both the citizen portal and the admin dashboard, which are gated by separate login flows.
- Supabase Storage handles report photo attachments (up to 5 images per report, 5MB each).
- Supabase's PostgreSQL change-data-capture (CDC) realtime subscriptions replace the custom synchronisation layer originally planned, and are what keep the citizen portal and the separate admin dashboard in sync.
- The system was implemented as two independent React applications, the citizen-facing TrackServ app and a separate admin-portal app, rather than the single application implied by the proposal, sharing the same Supabase project as their common backend.

This shift removes an entire custom backend tier (Node.js + PHP) in favour of a managed platform. It reduced implementation time and, notably, resulted in a simpler architecture than the one proposed but backend logic now lives largely in Supabase configuration and client-side calls rather than in an independently documented API layer, which is flagged as an outstanding item in Section 15.

3.2 Hardware and software limitations

- Client devices require a modern browser with JavaScript and geolocation API support.
- The system depends on Supabase's hosted infrastructure and availability rather than self-managed servers.
- Users must have location services enabled, search by name, or select a point on the map for accurate issue geotagging; reports outside the Tshwane Municipality boundary (defined by a GeoJSON polygon) are rejected at submission.
- Photo uploads are capped at five images per report and 5MB per file, and are served at full resolution with no visible compression or lazy loading.
- The citizen portal is currently deployed to GitHub Pages, which serves static files only; any future feature needing server-side processing (e.g. sending emails) would require a serverless function or separate backend.

3.3 Performance limitations

The system must handle real-time GPS data capture and provide immediate visual updates on both the citizen dashboard and the administrative map. Reliability is critical; the system must remain accessible to the community 24/7 so urgent issues can be reported at any time. Supabase Realtime subscriptions are active across multiple pages on both applications, which has not yet been load-tested at scale, and full-resolution image uploads combined with the absence of code-splitting or lazy loading are noted as performance risks worth addressing before wider rollout (Section 12).

4. Required skills

- Proficiency in JavaScript/React for both the citizen portal and the separately built administrative dashboard.
- Working knowledge of PostgreSQL, including schema design and row-level security policy authoring.
- Practical experience with Supabase: Auth, Storage, Realtime/CDC subscriptions, and the auto-generated API.
- Geolocation and mapping: GPS capture, reverse geocoding, GeoJSON boundary validation, and interactive map rendering with street/satellite toggling (OpenStreetMap and Esri).
- Charting/analytics library experience for both the citizen category breakdown and the admin analytics dashboard (trend charts, donut charts).
- Responsive, mobile-first UI design across defined breakpoints (480px, 640px, 768px, 1024px).

5. Design approach/methodology

5.1 Methodology

The project followed a standard Software Development Life Cycle (SDLC) using an iterative design (Spiral Model) approach, which allowed the administrative dashboard, assignment system, and analytics module to be added as separate, later iterations once the core citizen reporting loop was validated.

5.2 Project components

The project was delivered as two connected applications, across the following components:

- Citizen portal: guided issue reporting (category, description, photos, location), personal dashboard with statistics and an interactive map, community voting, badges, profile page, community social page, and a services market place.
- Administrative portal: operations map with street/satellite toggle, location filters, report management, staff assignment system, and an analytics module.
- Supabase project setup: database schema, authentication for both apps, storage buckets, and real-time subscriptions connecting the two.
- Geolocation mapping integration, including reverse geocoding and municipal boundary (GeoJSON) validation.
- Responsive, mobile-first UI across both applications.
- Testing phases (Security, Usability, and Performance).
- Maintenance and review.

5.3 Planned timeline

- Planning & Requirements Gathering: 1 week
- System & UI/UX Design: 2 – 3 weeks
- Citizen Portal Development: 3 weeks
- Administrative Dashboard Development (added scope): 2 weeks
- Community & Engagement Features (voting, badges, community page, services marketplace): 1–2 weeks (added scope)
- Implementation & Local Deployment: 2 weeks
- User Testing (30 Users & 5 NGOs): 2 days
- Post-Launch Maintenance & Review: 2 weeks

6. Architecture & Technology Decisions

The decision to move from a custom Node.js/PHP/MySQL stack to Supabase was made after weighing development speed against infrastructure control. A further architectural decision, not in the original proposal, was to build the administrative dashboard as a fully separate React application rather than an admin view within the citizen app. This gives municipal staff a distinct, security-conscious interface without exposing administrative routes inside the public-facing bundle.

- Both applications read from and write to the same Supabase project, so there is a single source of truth for reports.
- Real-time synchronisation uses Supabase's PostgreSQL change-data-capture subscriptions: a new citizen report appears on the admin map without a refresh, and an administrator's status change appears on the citizen dashboard without a refresh.
- Authentication is handled separately for each app via Supabase Auth, with the admin dashboard gated behind its own login.
- The trade-off is that access control currently relies on the application layer (requiring admin login) rather than being enforced with row-level security at the database layer, see Section 11.

7. Citizen Portal Features

7.1 Reporting an issue

Citizens report issues through a guided, five-part form: category (Roads & Infrastructure, Water Leak, Electricity, Garbage, or Other, each with an icon and colour code), a 500-character description, location, up to five photos (5MB each, via drag-and-drop or file picker), and optional additional information.

Location can be captured via device GPS, searched by name, or set by clicking the interactive map, with coordinates converted to a readable address through reverse geocoding. Submissions are validated against a GeoJSON boundary for Tshwane Municipality, and locations outside the boundary are rejected with a clear on-screen warning.

7.2 Tracking progress

The citizen dashboard opens with a hero section and four statistics cards (In Progress, Under Review, Resolved, and Community Fixes This Month), followed by the eight most recently submitted reports platform-wide, each showing status, time submitted, vote count, and a flame icon for top-voted issues. An interactive map, filterable by status, plots all reports with colour-coded markers, and a category breakdown pie chart shows the mix of issue types being reported.

7.3 Community voting and badges

Citizens can upvote an existing report instead of duplicating it; upvoted reports are highlighted and show a flame icon and vote count, giving administrators a crowdsourced signal of community priority. A badge system recognises participation for example, a first-report badge, a badge for frequent reporting, and a badge for consistently reporting environmental issues shown on the user's profile alongside their submitted and resolved report counts.

7.4 Responsive design

Layouts adapt across four breakpoints (480px, 640px, 768px, 1024px): maps and complex layouts stack vertically on mobile and arrange side by side on desktop, and the navigation sidebar collapses into a mobile menu. A shared component library (logo, footer, sidebar) keeps branding and behaviour consistent across pages.

8. Administrative Dashboard

The administrative dashboard was built as a separate application to give municipal staff an operational tool distinct from the citizen-facing portal, and represents scope beyond the original proposal.

8.1 Operations map and situational awareness

On login, administrators see a statistics panel (Total Locations, Active Reports, In Progress, Resolved, and Overdue open more than 7 days) above a live map of all reports, toggleable between street view (OpenStreetMap) and satellite view (Esri). Markers are colour-coded by status, with overdue reports additionally marked in red, and clicking a marker opens a popup with the report's title, location, status and reporter details.

8.2 Filtering and prioritisation

A Location Filters panel allows searching by title, location, or category, and filtering by category or status, with apply/reset controls. A Top Areas panel ranks locations by report frequency to help guide resource allocation, and a Recent Locations table lists the latest reports with colour-coded status badges, including an explicit "Overdue" badge. A Location Overview donut chart summarises the status distribution.

8.3 Report management

The Reports page lists all submitted reports with title, description, location, category, submission date, vote count and photo thumbnails, and supports filtering by category/status and searching, so staff can find, for example, all unresolved water issues from the last week.

8.4 Assignment system

The Assignments page was not present in the original proposal, it lets administrators delegate reports to specific team members with a priority level (Low, Medium, High) and optional notes, and lets supervisors filter by assignee, priority, category, or status to see workload distribution and identify bottlenecks.

8.5 Analytics

The Analysis page lets staff select a time range (7 days, 30 days, 90 days, or year-to-date) and view Reports Analysed, Average Resolution Time, Resolution Rate and Overdue Rate, each with a trend comparison against the previous period. A six-month trend chart compares reports received against reports resolved, a category breakdown donut chart shows the issue mix, and a team workload chart breaks down in-progress versus open reports by category.

8.6 Manual report entry

An Add Location function allows administrators to manually create a report title, description, location, category, severity and coordinates for issues raised through other channels (e.g. a council meeting or social media), with status defaulting to "open" so it enters the normal workflow.

9. Real-Time Synchronisation Between Portals

New citizen reports appear on the admin map and reports list without a manual refresh, and status changes made by an administrator (Open → In Progress → Under Review → Resolved/Closed) appear on the citizen's dashboard within seconds, using Supabase's PostgreSQL change-data-capture subscriptions rather than polling. When multiple administrators are working at once, each sees the others' changes including assignments propagate live, reducing the risk of duplicate or conflicting work. Overdue tracking (open more than 7 days) is highlighted on both the admin map/table and reflected in the analytics overdue rate, creating shared accountability.

This directly addresses the proposal's transparency goal: citizens who were previously "frequently left unaware of the progress or resolution status" now see status changes reflected live rather than after a delay.

10. Media & File Management

Photo evidence is handled through Supabase Storage rather than a self-managed file server, keeping media under the same authenticated project as the rest of the data. Uploads are currently stored at full resolution with no visible compression, resizing, or lazy loading, which is noted as a performance item to address before handling larger volumes of citizen-submitted photos (Section 12).

11. Security Assessment

Both applications are gated by Supabase Auth login, and both automatically log users out after 5 minutes of inactivity. This is an appropriate baseline, but a review of the implementation surfaced several gaps that should be closed before production deployment:

- Row-level security (RLS) policies at the database layer are tailored for each table in the database, access control currently relying on the database itself restricting which rows a given user or role can read or write.
- Form validation (email format, password length, report fields) is enforced client-side and database, which would stop a determined actor manipulating requests directly.
- No rate limiting on report submissions but votes a rate limit of one vote per user for an issues.

12. Performance & Scalability

- Supabase's managed PostgreSQL instance handles database scaling, replacing the need to provision and tune a self-hosted MySQL server.
- Realtime subscriptions are active across multiple pages on both apps; behaviour under many simultaneous connected sessions has not yet been tested.
- Full-resolution photo uploads and the absence of code-splitting or lazy loading are likely to slow load times as report volume grows, particularly for citizens on poor connections.
- No load testing has been performed against realistic data volumes (e.g. a dataset of a thousand reports across varied statuses and ages).

13. Testing & QA

Usability testing is to be carried out as originally planned, recruiting 30 potential community members and 5 local NGOs to perform typical tasks such as logging a new issue, checking a status update, and providing feedback, with performance measured via task completion time and error rates, supplemented by trust and satisfaction questionnaires.

14. Evaluation of solution

The success of TrackServ will be judged using a small-scale usability test. The project should recruit 30 potential community members and 5 local NGOs to perform 3–5 typical tasks (e.g., logging a new issue, checking a status update, providing feedback). Their performance will be measured via metrics such as task completion time and error rates, supplemented by questionnaires to gauge user trust and satisfaction with the platform's transparency.

15. Known Limitations & Future Work

The review identified the following gaps to prioritise before wider deployment:

- No automated test coverage on either the citizen portal or the admin dashboard.
- The platform is set to a single municipality (Tshwane): boundaries, default map location and branding are not yet configurable for reuse elsewhere.
- GitHub Pages hosting only serves static files, so any future feature needing server-side processing (e.g. email notifications) will need a serverless function or separate backend.

Future work, in rough priority order:

- Email notifications on status change, so citizens do not need to keep checking the dashboard.
- Generalise municipal boundaries and branding to support more than one municipality.
- Native mobile apps with push notifications, once the web platform's core gaps are closed.
- Longer-term: use accumulated resolution-time data to flag chronic problem areas or predict which report categories are at risk of becoming overdue.

16. Production Readiness Roadmap

Before proceeding to a full production rollout beyond the planned usability test, the following should be addressed:

- Load-test both applications against a realistic dataset (e.g. ~1,000 reports of varying status and age) to confirm the map, dashboard and analytics remain responsive.
- Define municipal operating policy: target resolution times per severity, escalation paths for stalled reports, and who has authority to reassign or close a report, alongside basic training materials for administrators and a short usage guide for citizens.

17. Conclusion

This revised proposal reflects a platform that has grown from a single reporting-and-tracking portal into a two-sided civic engagement system: a citizen portal and a dedicated administrative dashboard, kept in sync in real time on a shared Supabase backend. By adopting Supabase in place of a custom Node.js/PHP/MySQL stack, and by adding community voting, badges, community engagement page, services market, staff assignment and analytics, TrackServ goes well beyond its original scope in addressing the communication, transparency and data-accuracy problems it set out to solve. The most significant remaining gap is incomplete testing, and documentation, are the clear priorities before the platform can move from a feature-complete to a production-ready civic tool.

Software Deliverable Summary:

- Citizen portal: guided issue reporting, personal dashboard, community voting, badges, and profile management.
- Administrative dashboard: live operations map, filterable report management, staff assignment system, and analytics module.
- Shared Supabase backend (PostgreSQL, Auth, Storage) with real-time change-data-capture synchronisation between both applications.
- Geolocation mapping with reverse geocoding and municipal boundary validation.

Complexity/difficulty component(s) of the project:

- Designing and maintaining two separate applications against a single shared backend without a dedicated API layer.
- Implementing real-time, bidirectional data synchronisation between the citizen and administrative applications via Supabase CDC.
- Integrating GPS/geolocation capture, reverse geocoding, and municipal-boundary validation within a cross-browser web environment.
- Building a staff assignment and analytics system (resolution time, resolution rate, overdue tracking, workload by category) not scoped in the original proposal.
- Designing a voting and badge system that meaningfully reflects community priority without being easy to game.

References

Besedina, E., 2024. integrio systems. [Online]

Available at: <https://integrio.net/blog/how-to-choose-a-technology-stack-for-web-applications>

[Accessed 03 May 2024].

madiba, Erasmus, L. & Lawrence, D., 2023. researchspace. [Online]

Available at: <https://researchspace.csir.co.za/items/28b8430b-1b56-415e-b116-3ea0dff1205d>

[Accessed 03 May 2026].

Supabase, 2026. Supabase Documentation. [Online]

Available at: <https://supabase.com/docs>

[Accessed 10 August 2026].